Multiview Scene Estimation for the AIC Task Board

From parallel Gazebo data generation to a learned estimator of board and port poses · AI for Industry Challenge workspace · November 2026

> **Result.** A single learned model takes the three wrist camera images and returns the pose of the task board and of all 15 named ports in the robot base frame, with presence and confidence for each. On 1,994 held out simulated scenes the mean translation error is **1.3 mm for the board, 1.8 mm for SFP ports and 2.05 mm for SC ports**, with 97 to 100% of estimates within 5 mm and presence accuracy of 98.5% or better.
>
> The model was trained on 20,000 automatically labeled scenes. They were generated by six isolated Gazebo instances in about 2.5 hours, and training took 62 minutes on one RTX 5090.

1. Problem and scope

The AIC task is to insert cables into SFP ports on network cards and into SC ports on an optical panel, on a task board whose pose and component layout change every trial. Before any insertion, a task planner needs to know where each port is. This work builds that capability: an estimator that answers "*where is* `nic_card_mount_3/sfp_port_1`*, is it there, and how sure are you*" in the robot `base_link` frame, from the three cameras the challenge provides.

Version 1 is deliberately narrow. Images come from one observation pose (the robot HOME pose), the simulator provides exact calibration and privileged ground truth, and no cable is held in the gripper. The design keeps each of these as an explicit input or configuration so that later versions can relax them without restructuring.

2. System overview

Collect	→	Curate	→	Train	→	Evaluate	→	Infer
Parallel Gazebo containers spawn randomized boards and record images and ground truth.		Validate every scene, reject repeated frames and hidden boards, compute normalization.		Multiview transformer with one query per named target. Logged to W&B.		Per target and per type errors on the held out split.		ROS node publishes `SceneEstimate` and TF frames at 5 Hz.

Everything lives in one package, `aic_utils/aic_perception`, plus a small interface package for the output messages. Every stage is a script driven only by JSON configuration: a shared `base.json` (board schema, image size, dataset location) and one overlay per stage. Validation is strict, so a misspelled or missing key stops the run instead of silently using a default.

3. Data generation

Isolated, parallel simulation

Each collection worker is one copy of the official `aic_eval` container running the unmodified AIC bringup headless with ground truth enabled. Because each container has its own network namespace, every Gazebo server, Zenoh router and ROS graph is isolated without any domain or partition bookkeeping. Workers spawn boards through the same `/gz_server/spawn_entity` service the AIC engine uses. They wait for a synchronized `/observations` message rendered after the spawn, then read every port pose from the simulator's pose publisher, the same source AIC scoring uses.

Rendering decides throughput. With software rendering, one instance consumed about 18 CPU cores and ran at roughly 1% of real time. Routing OpenGL to the GPU through the WSL2 `/dev/dxg` device made each instance run in real time on about 1.6 cores. Six staggered workers sustain about **140 scenes per minute**. Beyond six concurrent instances the GPU driver path becomes unstable, so the orchestrator relaunches failed workers and collection resumes where it stopped.

Scene randomization and labels

- **Board:** position over a 15 × 35 cm region and any yaw.
- **Components:** one to five NIC cards with rail offset and ±10° yaw, zero to five SC ports with nonoverlapping rail offsets, and pick fixtures present with probability 0.5.
- **Determinism:** every scene is a pure function of (seed, scene ID), so any worker can regenerate any scene and the train or validation assignment is fixed at collection time.
- **Labels:** each target records whether it was spawned, its pose in `base_link`, and its projected pixel in each camera. It counts as *present* when its origin falls inside at least one image.

Dataset

20,000 scenes, 60,000 images at 576 × 512 (half the native 1152 × 1024), 2.8 GB on disk. The layout is flat per split, with one image folder (`scene_0000123_left.jpg`) and one label folder. After curation, 17,996 training and 1,994 validation scenes are usable; ten were dropped because the board was out of view. On average 8.6 ports are spawned per scene and 6.2 are visible from HOME.

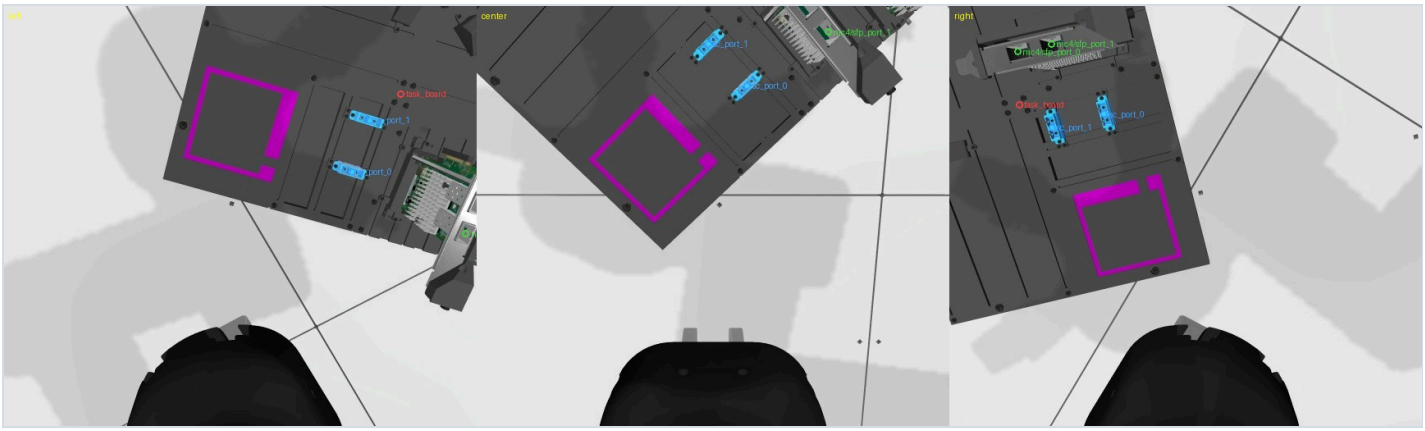


Figure 1. One curated scene from the left, center and right cameras with ground truth target origins projected onto the images (red: board, green: SFP ports, blue: SC ports). Curation writes previews like this for a visual label check before training.

4. Model

```
left / center / right RGB (576 × 512)
        │  shared ResNet-18 (ImageNet init)
        ▼
 18 × 16 feature tokens per camera
        +  ray embedding per token
           (origin, direction in base_link
            from intrinsics and TF pose)
        ▼
 transformer encoder, 3 layers
   (fuses all 864 tokens across views)
        ▼
 16 learned target queries
   → transformer decoder, 4 layers
        ▼
 per target: presence · xyz · 6D rotation
             · confidence
```

Named outputs. A versioned board schema (`aic_v1`) lists the 16 targets in a fixed order: the board, ten SFP ports and five SC ports. Query i always means target i, and planners only ever see target IDs. A different board means a new schema version and fine tuning, not new code.

Calibration as an input. Every image token carries the base frame ray through its pixel, computed from the camera matrix and the camera pose read from TF. Calibration enters as data rather than being memorized in the weights, which is what allows a moving camera or recalibrated robot later.

Losses. Presence uses binary cross entropy on every target. Pose uses L1 translation and chordal rotation, applied to present targets only. Confidence is trained to predict whether the current pose error is below 5 mm and 5°. The model has 18.0 M parameters, 11.2 M of them in the image encoder.

5. Data integrity: what went wrong and how it is now prevented

Three defects were found by auditing the data rather than trusting the training curve. Each one now has a guard at the point where it would enter the dataset.

Issue	Symptom and cause	Fix and guard
Frozen renderer	In 2 of 6 workers the GPU renderer stopped while the simulator kept publishing frames with fresh timestamps. 4,771 scenes (35%) carried one repeated empty image with correct new labels. The timestamp check could not see it.	Each capture is compared with the previous scene. Healthy consecutive scenes differ by at least 6 gray levels and frozen ones by exactly 0. Curation also rejects any frame repeated across scenes. Affected scenes were deleted and recollected.
Empty frame leak	A container that started with a broken renderer saved one empty scene before the consecutive check fired. 11 scenes were affected.	Each worker records the empty scene at startup, and a capture that should show the board but matches it is rejected.
Ambiguous SC identity	SC ports on a shared rail look identical, but their names were assigned in random order, so the images could not determine which name belonged to which port. Validation error was 25 mm and presence accuracy 83%.	SC names now mean rail order: `sc_port_k` is the kth port along its rail. Existing labels were renamed without recollection, the order was verified against measured positions in all 20,000 labels, and curation rejects any label out of rail order.

6. Results

Two full training runs are compared. Run 1 used the original SC naming and batch 16. Run 2 used rail ordered SC naming, batch 64 with square root scaled learning rates, and channels last convolutions. Both ran 60 epochs with AdamW and cosine decay, and the best checkpoint is chosen by validation port error.

Target	Run 1 mean	Run 2 mean	Run 2 median	Within 5 mm	Rotation	Presence
Board	2.70 mm	**1.30 mm**	1.22 mm	99.8%	0.63°	100%
SFP ports (10)	3.45 mm	**1.82 mm**	1.53 mm	97.6%	1.21°	99.4%
SC ports (5)	25.39 mm	**2.05 mm**	1.47 mm	96.6%	2.97°	98.5%
All ports	11.72 mm	**1.90 mm**				

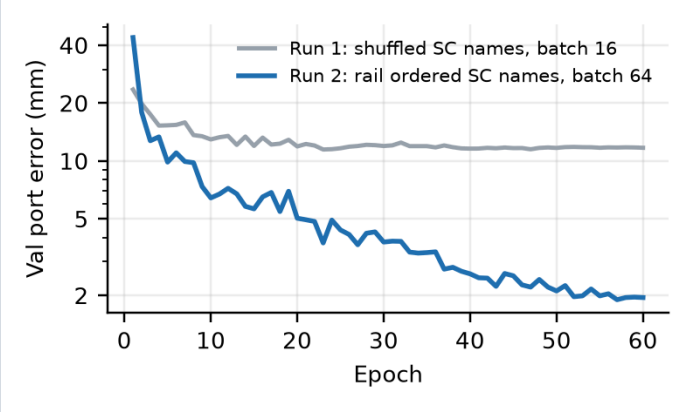


Figure 2. Validation port error per epoch. Run 1 plateaus near 12 mm because SC identity is not observable; Run 2 keeps improving to 1.9 mm.

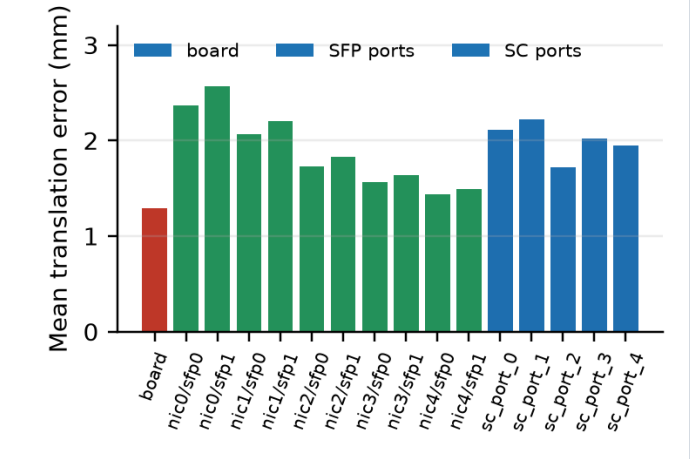


Figure 3. Run 2 mean translation error per target. SFP error rises toward rail 0, which is farthest from HOME and visible in only about 26% of scenes.

The SC fix accounts for most of the improvement, cutting SC error twelvefold and bringing every SC port into line with the SFP ports. Board and SFP error, whose labels did not change, also roughly halved; that gain comes from the larger batch and learning rate. Because the two runs changed both factors together, their individual contributions to the board and SFP gain are not separated. The remaining train to validation gap is small, about 1.1 mm against 1.9 mm.

Compute and cost

Training run 2 took 62 minutes. Batch 64 raised GPU utilization from about 80% to about 90% and cut epoch time from 91 s to 58 s, using 20.7 of 32.6 GB of GPU memory with no spill into shared memory. At about 0.65 kW for the whole machine, one training run uses roughly 0.7 kWh, about $0.12 at an average US residential rate. Collecting the dataset used a similar amount of energy.

7. Limitations

- **Simulation only:** results are on held out scenes from the same simulated distribution. Real cameras, lighting, materials and calibration error are not yet represented.
- **One viewpoint:** all images are from HOME, which sometimes sees only part of the board. The model receives camera poses as input, but it has never seen a different pose.
- **No cable in the gripper:** during the actual task a cable hangs in view and can hide ports.
- **Resolution:** features at 1/32 of a 576 × 512 image limit fine localization. The rail 0 trend suggests distance and viewing angle now dominate error.
- **Untested at task level:** the inference node has been verified against a live simulator, but no insertion has yet been driven from these estimates.

8. Next steps

- Close the loop: plan the far approach from `SceneEstimate` in the AIC engine and measure success rate, not only pose error.
- Collect from several observation poses and with the cable attached, so the camera pose input and occlusion handling are actually exercised.
- Add photometric, texture and calibration randomization ahead of any real camera test.
- Try finer features (stride 16 or higher input resolution) aimed at submillimeter error on distant ports.

Reproduce from the repository root with `pixi run --locked python aic_utils/aic_perception/scripts/<stage>.py` for `collect_data`, `curate_dataset`, `train`, `evaluate` and `run_inference`. Package README: `aic_utils/aic_perception/README.md`.